

MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

CYE 5153 Linux OS and Scripting Lab – 6.2

Reg. Number	Name	Branch
261100690010	Hegde Aansh Dilip	Cyber Security

MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Write an AWK program for each question. Ignore the header row where appropriate.

1. Print the source IP and event from every record.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 {print $2, $3}' events.log
192.168.1.10 Login_Success
192.168.1.25 Login_Failed
10.0.0.8 Port_Scan
192.168.1.25 Login_Failed
10.0.0.12 Malware_Alert
```

2. Display the time, source IP, and attempts in aligned columns using printf.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'BEGIN {printf "%-10s %-16s %8s\n", "Time", "SourceIP", "Attempts"} NR>1  {printf "%-10s %-16s %8d\n", $1, $2, $4}' events.log
Time       SourceIP         Attempts
09:10      192.168.1.10            0
09:12      192.168.1.25            0
09:15      10.0.0.8                0
09:18      192.168.1.25            0
09:22      10.0.0.12               0
```

3. Store the source IP and event in variables, then print them in a sentence.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 {ip=$2; ev=$3; print "Source " ip " generated the event " ev "."}' events.log
Source 192.168.1.10 generated the event Login_Success.
Source 192.168.1.25 generated the event Login_Failed.
Source 10.0.0.8 generated the event Port_Scan.
Source 192.168.1.25 generated the event Login_Failed.
Source 10.0.0.12 generated the event Malware_Alert.
```

4. Calculate the processing time for each record if one attempt takes 2 seconds.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 {print $2, $3, $4*2 " seconds"}' events.log
192.168.1.10 Login_Success 0 seconds
192.168.1.25 Login_Failed 0 seconds
10.0.0.8 Port_Scan 0 seconds
192.168.1.25 Login_Failed 0 seconds
10.0.0.12 Malware_Alert 0 seconds
```

5. Calculate the total number of attempts in the file.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 {sum += $4} END {print "Total attempts:", sum}' events.log
Total attempts: 0
```

6. Calculate the average number of attempts per record.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 {sum += $4; n++} END {if (n>0) print "Average attempts:", sum/n}' events.log
Average attempts: 0
```

7. Count the number of Login_Failed records.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 && $3=="Login_Failed" {c++} END {print "Login_Failed records:", c+0}' events.log
Login_Failed records: 2
```

8. Start with a review quota of 10 and decrease it by one for every HIGH or CRITICAL record. Print the remaining quota.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'BEGIN {quota=10}
     NR>1 && ($5=="HIGH" || $5=="CRITICAL") {quota--}
     END {print "Remaining quota:", quota}' events.log
Remaining quota: 10
```

9. Replace Port_Scan with Network_Scan in the event field and print every record in CSV format.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'BEGIN {OFS=","}
    NR>1 && $3=="Port_Scan" {$3="Network_Scan"}
    {$1=$1; print}' events.log

Time,Source_IP,Event,Severity,Attempts
09:10,192.168.1.10,Login_Success,LOW,1
09:12,192.168.1.25,Login_Failed,MEDIUM,3
09:15,10.0.0.8,Network_Scan,HIGH,12
09:18,192.168.1.25,Login_Failed,HIGH,7
09:22,10.0.0.12,Malware_Alert,CRITICAL,1
```

10. Add one to the attempts field of each Login_Failed record and print every record in CSV format.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'BEGIN {OFS=","}
     NR>1 && $3=="Login_Failed" {$4++}
     {$1=$1; print}' events.log

Time,Source_IP,Event,Severity,Attempts
09:10,192.168.1.10,Login_Success,LOW,1
09:12,192.168.1.25,Login_Failed,1,3
09:15,10.0.0.8,Port_Scan,HIGH,12
09:18,192.168.1.25,Login_Failed,1,7
09:22,10.0.0.12,Malware_Alert,CRITICAL,1
```

11. Print the source IP and event when severity is HIGH and attempts are greater than 5.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 && $5=="HIGH" && $4>5 {print $2, $3}' events.log
```

12. Print the source IP and event when severity is HIGH or CRITICAL.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 && ($5=="HIGH" || $5=="CRITICAL") {print $2, $3}' events.log
```

13. Use the conditional operator to label each record Repeated if attempts exceed 1; otherwise label it Single.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 {label = ($4>1) ? "Repeated" : "Single"; print $2, $3, $4, label}' events.log
192.168.1.10 Login_Success LOW Repeated
192.168.1.25 Login_Failed MEDIUM Repeated
10.0.0.8 Port_Scan HIGH Repeated
192.168.1.25 Login_Failed HIGH Repeated
10.0.0.12 Malware_Alert CRITICAL Repeated
```

14. Use the conditional operator to label each record Investigate if attempts are at least 5; otherwise label it Routine.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 {label = ($4>=5) ? "Investigate" : "Routine"; print $2, $3, $4, label}' events.log
192.168.1.10 Login_Success LOW Investigate
192.168.1.25 Login_Failed MEDIUM Investigate
10.0.0.8 Port_Scan HIGH Investigate
192.168.1.25 Login_Failed HIGH Investigate
10.0.0.12 Malware_Alert CRITICAL Investigate
```

15. Use if–else to print Review required for HIGH or CRITICAL records and Routine review for all other records.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 {
    if ($5=="HIGH" || $5=="CRITICAL")
        print $2, $3, $5, "Review required"
    else
        print $2, $3, $5, "Routine review"
}' events.log
192.168.1.10 Login_Success 1 Routine review
192.168.1.25 Login_Failed 3 Routine review
10.0.0.8 Port_Scan 12 Routine review
192.168.1.25 Login_Failed 7 Routine review
10.0.0.12 Malware_Alert 1 Routine review
```

16. Use if–else if–else to classify attempts as High activity (10 or more), Moderate activity (5–9), or Low activity (fewer than 5).

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 {
    if ($4 ≥ 10)     level = "High activity"
    else if ($4 ≥ 5) level = "Moderate activity"
    else             level = "Low activity"
    print $2, $3, $4, level
}' events.log
192.168.1.10 Login_Success LOW High activity
192.168.1.25 Login_Failed MEDIUM High activity
10.0.0.8 Port_Scan HIGH High activity
192.168.1.25 Login_Failed HIGH High activity
10.0.0.12 Malware_Alert CRITICAL High activity
```

17. Use a for loop to print Check 1 through Check 5.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'BEGIN {for (i=1; i≤5; i++) print "Check " i}'
Check 1
Check 2
Check 3
Check 4
Check 5
```

18. Use a while loop to count down from 10 to 1.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'BEGIN {i=10; while (i≥1) {print i; i--}}'
10
9
8
7
6
5
4
3
2
1
```

19. Use a do–while loop to count down from 10 to 1.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'BEGIN {i=10; do {print i; i--} while (i≥1)}'
10
9
8
7
6
5
4
3
2
1
```

20. Print each record's source IP, event, attempts, and activity classification in aligned columns. At the end, print the total records and total attempts.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'BEGIN {printf "%-16s %-14s %-9s %-18s\n", "SourceIP", "Event", "Attempts", "Activity"
          print "-------------------------------------------------------------"}
NR>1 {
    if ($4 >= 10)     level = "High activity"
    else if ($4 >= 5) level = "Moderate activity"
    else              level = "Low activity"
    printf "%-16s %-14s %-9d %-18s\n", $2, $3, $4, level
    records++; total += $4
}
END {print "-------------------------------------------------------------"
     print "Total records:", records
     print "Total attempts:", total}' events.log
SourceIP         Event          Attempts  Activity
-------------------------------------------------------------
192.168.1.10     Login_Success  0         High activity
192.168.1.25     Login_Failed   0         High activity
10.0.0.8         Port_Scan      0         High activity
192.168.1.25     Login_Failed   0         High activity
10.0.0.12        Malware_Alert  0         High activity
-------------------------------------------------------------
Total records: 5
Total attempts: 0
```

Login monitoring: A security analyst considers five or more failed attempts suspicious. Write an AWK script that prints the source IP and attempts for matching Login_Failed entries. At the end, print how many suspicious entries were found.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 && $3=="Login_Failed" && $4>=5 {print $2, $4; count++}
     END {print "Suspicious entries found:", count+0}' events.log
192.168.1.25 MEDIUM
192.168.1.25 HIGH
Suspicious entries found: 2
```

Incident summary: Write an AWK script that counts the entries in each severity category and prints a summary in an END block.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'NR>1 {count[$5]++}
     END {print "=== Incident Summary ==="
          for (s in count) printf "%-10s %d\n", s, count[s]}' events.log
=== Incident Summary ===
1          2
3          1
7          1
12         1
```

Processing estimate: Assume each attempt requires 2 seconds of analyst review. Calculate the review time for each entry and the total review time for the file. Use printf to present the results neatly.

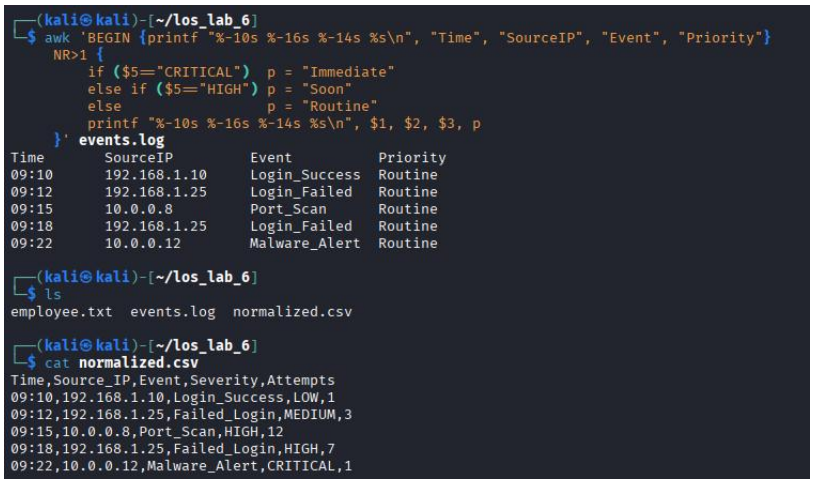
```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'BEGIN {printf "%-10s %-16s %-9s %s\n", "Time", "SourceIP", "Attempts", "Review(s)"}
     NR>1 {t = $4*2
           printf "%-10s %-16s %-9d %d\n", $1, $2, $4, t
           total += t}
     END {printf "\nTotal review time: %d seconds (%.2f minutes)\n", total, total/60}' events.log
Time       SourceIP         Attempts  Review(s)
09:10      192.168.1.10     0         0
09:12      192.168.1.25     0         0
09:15      10.0.0.8         0         0
09:18      192.168.1.25     0         0
09:22      10.0.0.12        0         0

Total review time: 0 seconds (0.00 minutes)
```

Log normalization: A monitoring tool requires the event name Failed_Login instead of Login_Failed. Update the matching field and print all records as CSV. The source file should remain unchanged.

```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'BEGIN {OFS=","}
     NR>1 && $3=="Login_Failed" {$3="Failed_Login"}
     {$1=$1; print}' events.log > normalized.csv
```

Priority queue: Assign each entry a priority: Immediate for CRITICAL, Soon for HIGH, and Routine for all others. Print the time, source IP, event, and assigned priority.



```
┌──(kali㉿kali)-[~/los_lab_6]
└─$ awk 'BEGIN {printf "%-10s %-16s %-14s %s\n", "Time", "SourceIP", "Event", "Priority"}
    NR>1 {
        if ($5=="CRITICAL")  p = "Immediate"
        else if ($5=="HIGH") p = "Soon"
        else                 p = "Routine"
        printf "%-10s %-16s %-14s %s\n", $1, $2, $3, p
    }' events.log
Time       SourceIP         Event          Priority
09:10      192.168.1.10     Login_Success  Routine
09:12      192.168.1.25     Login_Failed   Routine
09:15      10.0.0.8         Port_Scan      Routine
09:18      192.168.1.25     Login_Failed   Routine
09:22      10.0.0.12        Malware_Alert  Routine

┌──(kali㉿kali)-[~/los_lab_6]
└─$ ls
employee.txt  events.log  normalized.csv

┌──(kali㉿kali)-[~/los_lab_6]
└─$ cat normalized.csv
Time,Source_IP,Event,Severity,Attempts
09:10,192.168.1.10,Login_Success,LOW,1
09:12,192.168.1.25,Failed_Login,MEDIUM,3
09:15,10.0.0.8,Port_Scan,HIGH,12
09:18,192.168.1.25,Failed_Login,HIGH,7
09:22,10.0.0.12,Malware_Alert,CRITICAL,1
```